

Some Classical Attacks

Shibam Ghosh¹

June 5, 2026

¹Department of CSE, IIT Hyderabad

Table of contents

1. Feistel cipher with few interesting round functions
2. Meet-in-the-middle
3. Iterated Hash Functions
4. Multi-collisions in Hash functions

Feistel cipher with few interesting round functions

Feistel cipher

Definition (Encryption and Decryption)

Let $f_1, f_2, \dots, f_r$ be a round functions and let $k_1, \dots, k_r$ be sub-keys for the rounds $1, \dots, r$ respectively. Then to encrypt a message, split it into two equal parts: (L_0, R_0) and for each round $i = 0, 1, 2, \dots, n - 1$, compute

$$L_{i+1} = R_i$$

$$R_{i+1} = L_i + f_{i+1}(k_{i+1}, R_i)$$

The ciphertext is (R_n, L_n)

If the ciphertext is (R_n, L_n) , decryption is accomplished by computing for $i = n - 1, \dots, 1, 0$

$$R_i = L_{i+1}$$

$$L_i = R_{i+1} + f_{i+1}(k_{i+1}, R_i)$$

Round function $f_i(x) = 0$ for any x

What is the output of an r round feistel cipher when the input is (L_0, R_0) and each round function f_i outputs 0s for any input.

Let us compute few rounds

$$L_1 = R_0$$

$$R_1 = L_0 + 0 = L_0$$

$$L_2 = R_1 = L_0$$

$$R_2 = L_1 + 0 = L_1 = R_0$$

$$L_3 = R_2 = R_0$$

$$R_3 = L_2 + 0 = L_2 = L_0$$

Thus if r is even then (L_0, R_0) and if r is odd then (R_0, L_0) .

Round function $f_i(x) = 0$ for any x

What is the output of an r round feistel cipher when the input is (L_0, R_0) and each round function f_i outputs 0s for any input.

Let us compute few rounds

$$L_1 = R_0$$

$$R_1 = L_0 + 0 = L_0$$

$$L_2 = R_1 = L_0$$

$$R_2 = L_1 + 0 = L_1 = R_0$$

$$L_3 = R_2 = R_0$$

$$R_3 = L_2 + 0 = L_2 = L_0$$

Thus if r is even then (L_0, R_0) and if r is odd then (R_0, L_0) .

Round function f_i is an Identity function

What is the output of an r round feistel cipher when the input is (L_0, R_0) and each round function f_i is the identity function.

Let us compute few rounds

$$L_1 = R_0$$

$$R_1 = L_0 + f_1(k_1, R_0) = L_0 + R_0$$

$$L_2 = R_1 = L_0 + R_0$$

$$R_2 = L_1 + f_2(k_2, R_1) = L_1 + R_1 = R_0 + (L_0 + R_0) = L_0$$

$$L_3 = R_2 = L_0$$

$$R_3 = L_2 + f_3(k_3, R_2) = L_2 + R_2 = R_0$$

Thus after 3 rounds, we get the input back.

Round function f_i is an Identity function

What is the output of an r round feistel cipher when the input is (L_0, R_0) and each round function f_i is the identity function.

Let us compute few rounds

$$L_1 = R_0$$

$$R_1 = L_0 + f_1(k_1, R_0) = L_0 + R_0$$

$$L_2 = R_1 = L_0 + R_0$$

$$R_2 = L_1 + f_2(k_2, R_1) = L_1 + R_1 = R_0 + (L_0 + R_0) = L_0$$

$$L_3 = R_2 = L_0$$

$$R_3 = L_2 + f_3(k_3, R_2) = L_2 + R_2 = R_0$$

Thus after 3 rounds, we get the input back.

Round function f_i is an Identity function

What is the output of an r round feistel cipher when the input is (L_0, R_0) and each round function f_i is the identity function.

Let us compute few rounds

$$L_1 = R_0$$

$$R_1 = L_0 + f_1(k_1, R_0) = L_0 + R_0$$

$$L_2 = R_1 = L_0 + R_0$$

$$R_2 = L_1 + f_2(k_2, R_1) = L_1 + R_1 = R_0 + (L_0 + R_0) = L_0$$

$$L_3 = R_2 = L_0$$

$$R_3 = L_2 + f_3(k_3, R_2) = L_2 + R_2 = R_0$$

Thus after 3 rounds, we get the input back.

Round function f_i is XOR

What is the output of an r round feistel cipher when the input is (L_0, R_0) and each round function f_i is defined as $f_i(k, x) = x + k$

Let us compute few rounds

$$L_1 = R_0$$

$$R_1 = L_0 + f_1(k_1, R_0) = L_0 + R_0 + k_1$$

$$L_2 = R_1 = L_0 + k_1 + R_0$$

$$R_2 = L_1 + f_2(k_2, R_1) = R_0 + R_1 + k_2 = L_0 + k_1 + k_2$$

$$L_3 = R_2 = L_0 + k_1 + k_2$$

$$R_3 = L_2 + f_3(k_3, R_2) = L_2 + R_2 + k_3 = R_0 + k_2 + k_3$$

In general L_r and R_r is a linear function in L_0, R_0 and $k_1, k_2, \dots, k_r$.

Round function f_i is XOR

What is the output of an r round feistel cipher when the input is (L_0, R_0) and each round function f_i is defined as $f_i(k, x) = x + k$

Let us compute few rounds

$$L_1 = R_0$$

$$R_1 = L_0 + f_1(k_1, R_0) = L_0 + R_0 + k_1$$

$$L_2 = R_1 = L_0 + k_1 + R_0$$

$$R_2 = L_1 + f_2(k_2, R_1) = R_0 + R_1 + k_2 = L_0 + k_1 + k_2$$

$$L_3 = R_2 = L_0 + k_1 + k_2$$

$$R_3 = L_2 + f_3(k_3, R_2) = L_2 + R_2 + k_3 = R_0 + k_2 + k_3$$

Round function f_i is XOR

What is the output of an r round feistel cipher when the input is (L_0, R_0) and each round function f_i is defined as $f_i(k, x) = x + k$

Let us compute few rounds

$$L_1 = R_0$$

$$R_1 = L_0 + f_1(k_1, R_0) = L_0 + R_0 + k_1$$

$$L_2 = R_1 = L_0 + k_1 + R_0$$

$$R_2 = L_1 + f_2(k_2, R_1) = R_0 + R_1 + k_2 = L_0 + k_1 + k_2$$

$$L_3 = R_2 = L_0 + k_1 + k_2$$

$$R_3 = L_2 + f_3(k_3, R_2) = L_2 + R_2 + k_3 = R_0 + k_2 + k_3$$



In general L_r and R_r is a linear function in L_0, R_0 and $k_1, k_2, \dots, k_r$.

Meet-in-the-middle

Double Encryption

Consider two block ciphers $E_b : \{0, 1\}^m \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ and $E_a : \{0, 1\}^n \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$. Given a plaintext p and two encryption keys k_1 and k_2 , a ciphertext can be generated as,

$$c = E_b(k_2, E_a(k_1, p))$$

To decrypt a ciphertext c we apply the keys in reverse order as,

$$p = D_a(k_1, D_b(k_2, c)).$$

Let us consider several plaintext-ciphertext pairs $(p_1, c_1), (p_2, c_2), \dots, (p_t, c_t)$, which are obtained using the key (k_1, k_2) in double encryption. Then

$$E_a(k_1, p_i) = D_b(k_2, c_i) \text{ for all } 1 \leq i \leq t.$$

Double Encryption

Consider two block ciphers $E_b : \{0, 1\}^m \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$ and $E_a : \{0, 1\}^n \times \{0, 1\}^\ell \rightarrow \{0, 1\}^\ell$. Given a plaintext p and two encryption keys k_1 and k_2 , a ciphertext can be generated as,

$$c = E_b(k_2, E_a(k_1, p))$$

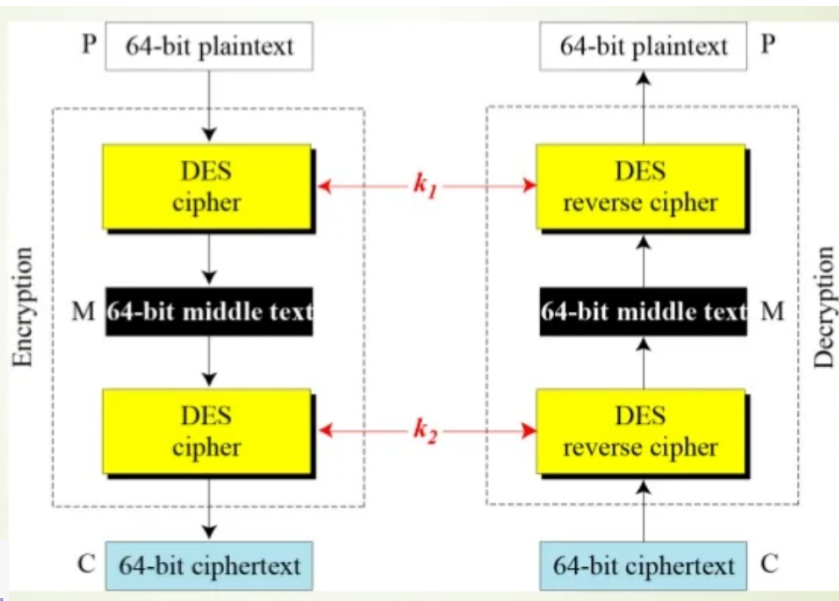
To decrypt a ciphertext c we apply the keys in reverse order as,

$$p = D_a(k_1, D_b(k_2, c)).$$

Let us consider several plaintext-ciphertext pairs $(p_1, c_1), (p_2, c_2), \dots, (p_t, c_t)$, which are obtained using the key (k_1, k_2) in double encryption. Then

$$E_a(k_1, p_i) = D_b(k_2, c_i) \text{ for all } 1 \leq i \leq t.$$

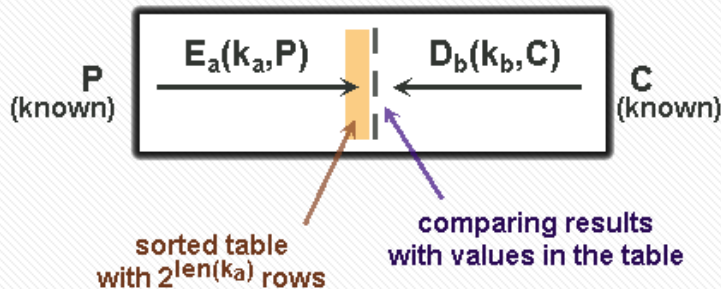
Double DES: Meet-in-the-middle



Double encryption: Meet-in-the-middle

Consider a plaintext-ciphertext pair (p_1, c_1) .

1. Construct a table T_1 with rows of the form (c, k_a) , where $E_a(k_a, p_1) = c$.
2. Sort the table T_1 by first element.
3. For every k_b , search for $D_b(k_b, c_1)$ in T_1 .
4. If there is a match and k_a is the second element, then we have a candidate key (k_a, k_b)



Double Encryption: Meet-in-the-middle

Complexity of the attack

Time complexity: $n2^m$

Space complexity: $\ell 2^n$

1. 2^m : Compute D_{k_b} for every k_b
2. $n2^m$: To sort the table T_1 .
3. $2^n(n + \ell)$: Size of the table T_1

Double DES: Meet-in-the-middle

Meet-in-the-middle attack exploits the equation $E_a(k_1, p_i) = D_b(k_2, c_i)$ which holds for any (p_i, c_i) .

What is the expected number of keys (k_1, k_2) such that $E_a(k_1, p_i) = D_b(k_2, c_i)$ holds for t pairs of (p_i, c_i) ?

1. Suppose we fix p and choose a random k_1 . Then $\Pr[E_a(k_1, p) = z] = 2^{-|k_1|}$ for all z .
2. Similarly, if we fix c and choose a random k_2 . Then $\Pr[D_b(k_2, c) = z] = 2^{-|k_2|} = 2^{-|k_1|}$ for all z (assume $|k_1| = |k_2|$).
3. Thus $\Pr[E_a(k_1, p) = D_b(k_2, c)] = 2^{-|k_1|}$.
4. If we consider t pairs, then the above equation holds for all pairs with probability $2^{-t|k_1|}$.
5. As there are total $2^{|k_1|}$ possible keys, the expected number of keys is $2^{|k_1| - t|k_1|}$.

Modified DES: Meet-in-the-middle

Say the key schedule of DES is modified as follows: the left half of the master key is used to derive all the sub-keys in rounds 1–8, while the right half of the master key is used to derive all the sub-keys in rounds 9–16.

1. Use the encryption oracle to obtain plaintext-ciphertext pair $(p_1, c_1), (p_2, c_2), (p_3, c_3)$.
2. Construct a sorted table T_1 with rows of the form (c, k_a) , where $E'_{k_a}(p_1) = c$. Here E' is 8-round DES encryption.
3. Similarly, construct another sorted table T_2 with rows of the form (p, k_b) , where $D'_{k_b}(c_1) = p$. Here D' is 8-round DES decryption.
4. If there is a match, then we have a candidate key (k_a, k_b)
5. We can use other two pairs for verification.

Modified DES: Meet-in-the-middle

Say the key schedule of DES is modified as follows: the left half of the master key is used to derive all the sub-keys in rounds 1–8, while the right half of the master key is used to derive all the sub-keys in rounds 9–16.

1. Use the encryption oracle to obtain plaintext-ciphertext pair $(p_1, c_1), (p_2, c_2), (p_3, c_3)$.
2. Construct a sorted table T_1 with rows of the form (c, k_a) , where $E'_{k_a}(p_1) = c$. Here E' is 8-round DES encryption.
3. Similarly, construct another sorted table T_2 with rows of the form (p, k_b) , where $D'_{k_b}(c_1) = p$. Here D' is 8-round DES decryption.
4. If there is a match, then we have a candidate key (k_a, k_b)
5. We can use other two pairs for verification.

Attack on Triple Encryption

Let f be an ℓ -bit block cipher. Define Triple Encryption as

$$F_{k_1, k_2}(p) = f_{k_1}(f_{k_2}^{-1}(f_{k_1}(p)))$$

1. Suppose for any (m_1, m_2) , it is possible to find in constant time all keys k_2 such that $m_2 = f_{k_2}^{-1}(m_1)$.
 - 1.1 Can you recover the full key of F_{k_1, k_2} using three plaintext-ciphertext pair?
 - 1.2 What is the time complexity?

Attack on Triple Encryption

1. Let $(p_1, c_1), (p_2, c_2), (p_3, c_3)$ be three collected plaintext-ciphertext pairs.
2. Initialize a set $K_g = \phi$. Then for every key k_a do the followings:
 - 2.1 Compute $m_1 = f_{k_a}(p_1)$ and $m_2 = f_{k_a}^{-1}(c_1)$
 - 2.2 Find the set S of all keys k_b such that $m_2 = f_{k_b}^{-1}(m_1)$ (it takes constant time).
 - 2.3 For every $k_b \in S$ do: if $c_2 = F_{k_a, k_b}(p_2)$ and $c_3 = F_{k_a, k_b}(p_3)$ then
 $K_g = K_g \cup \{(k_a, k_b)\}$.

Attack on Triple Encryption

Let us compute the running time.

1. The outer loop runs for $2^{|k_1|}$
2. Running time of the loop is linear in $|S|$
3. We expect that in each loop, only one key is in S , so total running time is 2^n
4. What is the probability that an incorrect key will be in K_g ? (2^{-n})

However, finding k_2 in constant time may not be possible. Using 2^n preprocessing time we can construct a table T containing rows of the form (m_2, k_2) where $m_2 = f_{k_2}^{-1}(0^n)$ and sort the table by m_2

Attack on Triple Encryption

Let us compute the running time.

1. The outer loop runs for $2^{|k_1|}$
2. Running time of the loop is linear in $|S|$
3. We expect that in each loop, only one key is in S , so total running time is 2^n
4. What is the probability that an incorrect key will be in K_g ? (2^{-n})

However, finding k_2 in constant time may not be possible. Using 2^n preprocessing time we can construct a table T containing rows of the form (m_2, k_2) where $m_2 = f_{k_2}^{-1}(0^n)$ and sort the table by m_2

Attack on Triple Encryption

We can use the table T as follows: First collect two plaintext-ciphertext pairs $(p_2, c_2), (p_3, c_3)$. Initialize a set $K_g = \phi$. Then for every key k_a do the followings:

1. Compute $p_1 = f_{k_a}^{-1}(0^n)$ and ask for an encryption of p_1 to F_{k_1, k_2} ; Let the result be c_1
2. Compute $m_1 = f_{k_a}(p_1)$ and $m_2 = f_{k_a}^{-1}(c_1)$
3. From table T , we find the row with first element m_2 and we consider the second element.
4. If the second element is k_b , do: if $c_2 = F_{k_a, k_b}(p_2)$ and $c_3 = F_{k_a, k_b}(p_3)$ then $K_g = K_g \cup \{(k_a, k_b)\}$.

What is the complexity of the whole attack?

Iterated Hash Functions

Hash Functions

1. A cryptographic hash function takes a “long” input string and produces a random looking ”short“ output string of fixed length.

$$\mathcal{H} : \{0, 1\}^* \rightarrow \{0, 1\}^n.$$

Security Properties

1. **Collision resistance:** Difficult to find $x_1, x_2 \in \{0, 1\}^*$ such that $x_1 \neq x_2 \wedge \mathcal{H}(x_1) = \mathcal{H}(x_2)$
2. **Preimage resistance:** Given $h \in \{0, 1\}^n$, it is difficult to find $x \in \{0, 1\}^*$ such that $\mathcal{H}(x) = h$
3. **Second Preimage resistance:** Given $x_1 \in \{0, 1\}^*$, it is difficult to find $x_2 \in \{0, 1\}^*$ such that $x_1 \neq x_2 \wedge \mathcal{H}(x_1) = \mathcal{H}(x_2)$.

Iterated Hash Function

Iterated Hash Function

1. Given a compression function $C : \{0, 1\}^{n+m} \rightarrow \{0, 1\}^n$ and some padding function.
2. **Preprocessing step:** Given a message $x \in \{0, 1\}^*$, construct a padded string $y = y_1 \| y_2 \| \cdots \| y_\ell$.
3. **Processing step:** Compute iterated hash output:

$$z_0 \leftarrow IV$$

$$z_1 \leftarrow C(z_0 \| y_1)$$

$$z_1 \leftarrow C(z_0 \| y_1)$$

$$\vdots$$

$$z_\ell \leftarrow C(z_{\ell-1} \| y_\ell).$$

4. Output $\mathcal{H}(x) = z_\ell$.

Multi-collisions in Hash functions

Multi-collisions in Hash functions

Multi-collisions in Hash functions

1. **t -multi-collision:** The problem is to find t different $x_1, x_2, \dots, x_t \in \{0, 1\}^*$ such that $\mathcal{H}(x_1) = \mathcal{H}(x_2) = \dots = \mathcal{H}(x_t) = y$.

The common output value, y , is called the *collision value*.

2. Is it harder to find t -multi-collision than one collision?

2.1 When iterated hash functions are involved, this intuition is false.

1. Suppose we have an algorithm A that can takes one chaining value z as input and finds two block $B^0 \neq B^1$ such that

$$c(z \| B^0) = c(z \| B^1).$$

2. We use this A to find multiple collisions.

Multi-collisions in Hash functions

Multi-collisions in Hash functions

1. **t -multi-collision**: The problem is to find t different $x_1, x_2, \dots, x_t \in \{0, 1\}^*$ such that $\mathcal{H}(x_1) = \mathcal{H}(x_2) = \dots = \mathcal{H}(x_t) = y$.

The common output value, y , is called the *collision value*.

2. Is it harder to find **t -multi-collision** than one **collision**?

2.1 When **iterated** hash functions are involved, this intuition is false.

1. Suppose we have an algorithm A that can takes one chaining value z as input and finds two block $B^0 \neq B^1$ such that

$$c(z \| B^0) = c(z \| B^1).$$

2. We use this A to find multiple collisions.

Multi-collisions in Hash functions

Multi-collisions in Hash functions

1. **t -multi-collision**: The problem is to find t different $x_1, x_2, \dots, x_t \in \{0, 1\}^*$ such that $\mathcal{H}(x_1) = \mathcal{H}(x_2) = \dots = \mathcal{H}(x_t) = y$.

The common output value, y , is called the *collision value*.

2. Is it harder to find **t -multi-collision** than one **collision**?
 - 2.1 When **iterated** hash functions are involved, this intuition is false.

1. Suppose we have an algorithm A that can takes one chaining value z as input and finds two block $B^0 \neq B^1$ such that

$$c(z \| B^0) = c(z \| B^1).$$

2. We use this A to find multiple collisions.

Multi-collisions in Hash functions

Multi-collisions in Hash functions

1. **t -multi-collision**: The problem is to find t different $x_1, x_2, \dots, x_t \in \{0, 1\}^*$ such that $\mathcal{H}(x_1) = \mathcal{H}(x_2) = \dots = \mathcal{H}(x_t) = y$.

The common output value, y , is called the *collision value*.

2. Is it harder to find **t -multi-collision** than one **collision**?
 - 2.1 When **iterated** hash functions are involved, this intuition is false.

1. Suppose we have an algorithm A that can takes one chaining value z as input and finds two block $B^0 \neq B^1$ such that

$$c(z \| B^0) = c(z \| B^1).$$

2. We use this A to find multiple collisions.

Multi-collisions in Hash functions

Multi-collisions in Hash functions

1. ***t*-multi-collision**: The problem is to find t different $x_1, x_2, \dots, x_t \in \{0, 1\}^*$ such that $\mathcal{H}(x_1) = \mathcal{H}(x_2) = \dots = \mathcal{H}(x_t) = y$.

The common output value, y , is called the *collision value*.

2. Is it harder to find ***t*-multi-collision** than one **collision**?

2.1 When **iterated** hash functions are involved, this intuition is false.

1. Suppose we have an algorithm A that can takes one chaining value z as input and finds two block $B^0 \neq B^1$ such that

$$\mathcal{C}(z \| B^0) = \mathcal{C}(z \| B^1).$$

2. We use this A to find multiple collisions.

Multi-collisions in Hash functions

Attack Algorithm

The attack works as follows:

1. Set $z_0 = IV$
2. for $i = 1$ to t :
 - 2.1 Call A to find $B_i^0 \neq B_i^1$ such that $\mathcal{C}(z_{i-1} \| B_i^0) = \mathcal{C}(z_{i-1} \| B_i^1)$
 - 2.2 let $z_i = \mathcal{C}(z_{i-1}, B_i^0)$
3. Pad (if necessary) and output 2^t messages of the form

$$(B_1^{b_1} \| B_2^{b_2} \| \dots \| B_t^{b_t}) \text{ where each } b_i \in \{0, 1\}$$



Complexity:

Running A for t times. So expected complexity $t \times 2^{n/2}$

Multi-collisions in Hash functions

Attack Algorithm

The attack works as follows:

1. Set $z_0 = IV$
2. for $i = 1$ to t :
 - 2.1 Call A to find $B_i^0 \neq B_i^1$ such that $\mathcal{C}(z_{i-1} \| B_i^0) = \mathcal{C}(z_{i-1} \| B_i^1)$
 - 2.2 let $z_i = \mathcal{C}(z_{i-1}, B_i^0)$
3. Pad (if necessary) and output 2^t messages of the form

$$(B_1^{b_1} \| B_2^{b_2} \| \dots \| B_t^{b_t}) \text{ where each } b_i \in \{0, 1\}$$



Complexity:

Running A for t times. So expected complexity $t \times 2^{n/2}$

Multi-collisions in Hash functions

Attack Algorithm

The attack works as follows:

1. Set $z_0 = IV$
2. for $i = 1$ to t :
 - 2.1 Call A to find $B_i^0 \neq B_i^1$ such that $C(z_{i-1} \| B_i^0) = C(z_{i-1} \| B_i^1)$
 - 2.2 let $z_i = C(z_{i-1}, B_i^0)$
3. Pad (if necessary) and output 2^t messages of the form

$$(B_1^{b_1} \| B_2^{b_2} \| \dots \| B_t^{b_t}) \text{ where each } b_i \in \{0, 1\}$$



Complexity:

Running A for t times. So expected complexity $t \times 2^{n/2}$

Cascaded Hash Function

Cascading

1. A natural way to build large hash values is to concatenate several smaller hashes.
2. Consider two hash functions

$$\mathcal{F} : \{0, 1\}^* \rightarrow \{0, 1\}^n$$

$$\mathcal{G} : \{0, 1\}^* \rightarrow \{0, 1\}^m$$

Construct the cascaded hash $\mathcal{H}(x) = \mathcal{F}(x) \parallel \mathcal{G}(x)$

3. What is the security of $\mathcal{H}$?
4. Output size of $\mathcal{H}$ is $(m + n)$, so using birthday attack the complexity will be $2^{\frac{m+n}{2}}$.
5. Can we do better?

Cascaded Hash Function

Cascading

1. A natural way to build large hash values is to concatenate several smaller hashes.
2. Consider two hash functions

$$\mathcal{F} : \{0, 1\}^* \rightarrow \{0, 1\}^n$$

$$\mathcal{G} : \{0, 1\}^* \rightarrow \{0, 1\}^m$$

Construct the cascaded hash $\mathcal{H}(x) = \mathcal{F}(x) \parallel \mathcal{G}(x)$

3. What is the security of $\mathcal{H}$?
4. Output size of $\mathcal{H}$ is $(m + n)$, so using birthday attack the complexity will be $2^{\frac{m+n}{2}}$.
5. Can we do better?

Cascaded Hash Function

Attack on Cascaded Hash Function

1. Without the loss of generality, assume $n < m$.
2. Using multi-collision idea, find $2^{m/2}$ -multi-collisions on $\mathcal{F}$. The complexity of this step is $\frac{m}{2} \times (2^{n/2})$.
3. Now we have $2^{m/2}$ messages with collision value, say f on the $\mathcal{F}$ side.

The hash values are of the form

$$x_1 \longrightarrow f \parallel g_1$$

$$x_2 \longrightarrow f \parallel g_2$$

$$\vdots$$

$$x_{2^{m/2}} \longrightarrow f \parallel g_{2^{m/2}}$$

Cascaded Hash Function

Attack on Cascaded Hash Function

1. Without the loss of generality, assume $n < m$.
2. Using multi-collision idea, find $2^{m/2}$ -multi-collisions on $\mathcal{F}$. The complexity of this step is $\frac{m}{2} \times (2^{n/2})$.
3. Now we have $2^{m/2}$ messages with collision value, say f on the $\mathcal{F}$ side.

The hash values are of the form

$$x_1 \longrightarrow f \parallel g_1$$

$$x_2 \longrightarrow f \parallel g_2$$

$$\vdots$$

$$x_{2^{m/2}} \longrightarrow f \parallel g_{2^{m/2}}$$

Cascaded Hash Function

Attack on Cascaded Hash Function

1. Now according to birthday paradox, these $2^{m/2}$ messages yields collision on the side of $\mathcal{G}$ also.
2. We need $2^{m/2}$ calls to $\mathcal{G}$.
3. Total complexity $\frac{m}{2} \times (2^{n/2}) + 2^{m/2}$.

Thank You for your attention!
Any questions?